Map

In **Go (Golang)**, a **map** is a built-in **hash table data structure** used to store **key–value pairs**. It is similar to dictionaries in Python or HashMap in Java.

Maps are **reference types** and dynamically grow as you add elements.

---

1. Basic Map Declaration

var m map[string]int

Explanation:

- map → built-in data structure
- string → key type
- int → value type

But this **does not allocate memory**. The map is **nil**.

If you try to insert values:

m["a"] = 1

You will get:

panic: assignment to entry in nil map

---

2. Creating a Map

Using make()

m := make(map[string]int)

Now the map is initialized.

Example:

package main

```
import "fmt"

func main() {

   m := make(map[string]int)

   m["apple"] = 10
   m["banana"] = 20

   fmt.Println(m)
}
```

Output

```
map[apple:10 banana:20]
```

---

3. Map Literal Initialization

```
m := map[string]int{
   "apple":  10,
   "banana": 20,
   "orange": 30,
}
```

Example

```
package main
import "fmt"
```

```
func main() {

   fruits := map[string]int{
      "apple":  10,
      "banana": 20,
   }

   fmt.Println(fruits)
}
```

---

4. Accessing Map Values

value := m["apple"]

Example

fmt.Println(m["apple"])

Output

10

---

5. Checking if Key Exists

Go provides **two return values**.

value, ok := m["apple"]

- value → stored value
- ok → boolean (key exists or not)

Example

```
v, ok := m["apple"]


if ok {
   fmt.Println("Exists:", v)
} else {
   fmt.Println("Key not found")
}
```

---

6. Deleting From Map

Use delete() function.

```
delete(m, "apple")
```

Example

```
delete(m, "banana")
```

---

7. Iterating Over Map

Use **range**.

```
for key, value := range m {
   fmt.Println(key, value)
}
```

Example

```
package main
import "fmt"
```

```go
func main() {

    m := map[string]int{
        "apple":  10,
        "banana": 20,
        "orange": 30,
    }

    for k, v := range m {
        fmt.Println(k, v)
    }
}
```

⚠️ **Important**

Map iteration order is **random**.

---

8. Map Length

len(m)

Example

```go
fmt.Println(len(m))
```

---

9. Map With Struct

Maps often store **structs**.

Example

```
type User struct {
   Name string
   Age  int
}

func main() {

   users := map[int]User{
      1: {"Alice", 25},
      2: {"Bob", 30},
   }

   fmt.Println(users[1])
}
```

---

10. Map With Slice

Example

```
m := map[string][]int{
   "a": {1,2,3},
   "b": {4,5,6},
}
```

---

11. Map as Reference Type

Maps are **reference types**, meaning copying a map copies the **pointer**, not the data.

Example

```
package main

import "fmt"


func main() {


   m1 := map[string]int{"a":1}


   m2 := m1


   m2["a"] = 100


   fmt.Println(m1)
}
```

Output

```
map[a:100]
```

Both variables reference the **same map**.

---

12. Nil Map Behavior

```
var m map[string]int
```

Valid operations:

read → allowed

write → panic

Example

```
fmt.Println(m["a"])  // 0
```

But

```
m["a"] = 1
```

panic.

---

13. Map Internals (Important for Deep Understanding)

A Go map internally contains:

```
map
 ├── bucket array
 │    ├── key
 │    ├── value
 │    └── hash
 └── overflow buckets
```

Process:

key → hash function → bucket index → value lookup

Features:

- average **O(1)** lookup
- uses **hashing**
- automatic **rehashing during growth**

---

14. Map Capacity Hint

m := make(map[string]int, 100)

This allocates **space for 100 elements**.

---

15. Map of Maps (Nested Map)

Example

```
users := map[string]map[string]int{
   "alice": {
      "math": 90,
      "eng":  85,
   },
}
```

Access

users["alice"]["math"]

---

16. Map CRUD Example

```
package main
import "fmt"

func main() {

   users := make(map[int]string)
```

```
    // Create
    users[1] = "Alice"
    users[2] = "Bob"

    // Read
    fmt.Println(users[1])

    // Update
    users[1] = "Charlie"

    // Delete
    delete(users, 2)

    fmt.Println(users)
}
```

---

17. Map Concurrency Warning

Maps are **NOT safe for concurrent access**.

This will crash:

fatal error: concurrent map writes

Use:

- sync.Mutex

- sync.RWMutex
- sync.Map

---

18. sync.Map (Concurrent Map)

Example

```
var m sync.Map

m.Store("a", 1)

v, ok := m.Load("a")
```

---

19. Map Memory Layout (Runtime)

At runtime:

```
hmap struct
 ├── count
 ├── buckets
 ├── oldbuckets
 ├── hash seed
 └── overflow buckets
```

This is implemented in:

runtime/map.go

---

20. When to Use Maps

Use maps when you need:

- key → value lookup
- fast search
- caching
- indexing
- counting frequency

Example:

word count

user id lookup

config maps